Carto Smart Shopping Cart System

Complete Project Overview, Features, Architecture, Technologies, and Setup

Generated from local project: C:\Users\LOQ\Downloads\CartoMAIN

1. Executive Summary

Carto is a full-stack smart shopping cart web application built with Next.js App Router. It lets a shopper create reusable grocery lists, link a selected list to a physical cart by QR code, track an active shopping session, view a live virtual receipt, and complete a demo checkout flow.
The project supports authenticated users and guest mode. It includes database-backed shopping lists, cart sessions, receipts, notifications, product search, checkout history, profile/dashboard screens, and PWA/mobile-ready behavior.

2. Technology Stack

- Framework: Next.js 14 App Router with React 18.
- Language: TypeScript.
- Database: PostgreSQL accessed through Prisma ORM.
- Authentication: NextAuth.js credentials provider with JWT sessions.
- State management: Zustand for active session state.
- Styling: Tailwind CSS with reusable UI components.
- QR and camera: @zxing/browser for scanning and qrcode/qrcode.react for QR generation.
- Payments: Mock checkout flow with Stripe dependency ready for integration.
- PWA: next-pwa service worker and manifest configuration.
- Validation: Zod schemas for user input and API payloads.

3. Core User Features

- Sign up and sign in with email/password.
- Continue as guest using local guest session data.
- Dashboard with quick actions, metrics, recent lists, and shopping entry points.
- Create, view, edit, and delete shopping lists.
- Add list items manually or from product search.
- Change item quantities, mark items collected, and delete items.
- Activate a shopping list and link it to a cart by scanning a QR code.
- Manual cart ID and pairing code entry for fallback cart linking.
- Active session screen showing cart status, progress, remaining items, collected items, and estimated total.
- Virtual receipt that shows scanned items, subtotal, tax, and total.
- Finish shopping, lock receipt, review checkout, and complete payment.
- Checkout success and paid receipt history.
- Profile and user-facing navigation on mobile and desktop.
- Notifications and user stats API support.

- Wishlist and favorite product API support.

4. Important UX and Performance Behavior

- List item actions use immediate feedback and optimistic UI updates.
- Buttons prevent duplicate submissions during async actions.
- Product search is debounced and aborts stale requests.
- Session and receipt polling avoid overlapping requests.
- API responses are trimmed with Prisma select where practical.
- Authenticated API routes are marked dynamic to avoid static-generation noise.
- Camera scanning warns users when HTTPS is required on phones.

5. Application Architecture

The app uses Next.js App Router. Page routes live under app, reusable visual elements live in components, server API endpoints live under app/api, Prisma schema and migrations live under prisma, shared helpers live in lib, Zustand stores live in store, and shared TypeScript interfaces live in types.

- Server components are used for page-level data loading where appropriate.
- Client components handle forms, camera access, optimistic UI, polling, and interactive controls.
- API routes enforce ownership through session checks and userId filtering.
- Prisma maps application models to PostgreSQL tables.
- Guest mode uses a local JSON-backed guest store for development/demo use.

6. Page Routes

- /
- /auth/signin
- /auth/signup
- /checkout
- /checkout/success
- /dashboard
- /history
- /lists
- /lists/:id
- /lists/new
- /profile
- /session
- /session/start

7. API Routes

- /api/auth/:...nextauth
- /api/auth/guest
- /api/auth/guest-bypass
- /api/auth/signup
- /api/cart/link

- /api/cart/qrcode
- /api/carts/:cartCode
- /api/lists
- /api/lists/:id
- /api/lists/:id/items
- /api/lists/:id/items/:itemId
- /api/notifications
- /api/payment/create
- /api/products
- /api/receipts/:id
- /api/receipts/:id/items
- /api/receipts/:id/items/:itemId
- /api/sessions
- /api/sessions/:id
- /api/sessions/:id/finish
- /api/sessions/active
- /api/stores
- /api/users/favorites
- /api/users/stats
- /api/wishlist
- /api/wishlist/:itemId

8. Prisma Data Model

The database schema covers users, stores, physical carts, shopping lists, list items, cart sessions, receipts, receipt items, products, user stats, favorite products, wishlists, and notifications.

- User
- Store
- Cart
- ShoppingList
- ListItem
- CartSession
- Receipt
- ReceiptItem
- Product
- UserStats
- UserFavoriteProduct
- Wishlist
- WishlistItem
- Notification

9. Shopping List Flow

Users create named lists, add products, update quantities, mark items collected, and activate a list when they are ready to connect to a smart cart. The list detail screen is optimized for repeated small actions and immediate feedback.

- Create list: POST /api/lists.
- Fetch list detail: app/lists/:id with list ownership checks.
- Add item: POST /api/lists/:id/items.
- Update item: PUT /api/lists/:id/items/:itemId.
- Delete item: DELETE /api/lists/:id/items/:itemId.

10. Cart Linking and QR Flow

A shopper selects a list, opens the connect-to-cart screen, scans a Carto QR code, validates the JSON payload, and posts cart data to /api/cart/link. The backend verifies list ownership, validates cart availability and pairing code, creates or reuses the physical cart, closes previous active sessions for the user, creates a new cart session and draft receipt, and creates a notification.

- Camera scanning uses browser media APIs and requires HTTPS on phones.
- Manual cart ID and pairing code are available when scanning is unavailable.
- The API hides sensitive pairing code data before responding.

11. Active Session and Receipt Flow

The active session page polls the session endpoint for cart/list/receipt changes and uses Zustand to hold the current session state. The receipt panel displays scanned items, subtotal, estimated tax, and total.

- GET /api/sessions/active finds the newest active or disconnected session for the user.
- GET /api/sessions/:id fetches one session with selected shopping list and receipt fields.
- POST /api/sessions/:id/finish completes the cart session and locks or creates a receipt.

12. Checkout and Payment Flow

Checkout loads the locked receipt, lets the user select a demo payment method, and posts to /api/payment/create. The payment route marks payment processing, completes the mock payment, marks the receipt paid, checks out the session, releases the cart, updates user stats, tracks favorites, creates a notification, and returns a payment result.

- Mock payment ID format: pi_mock_<timestamp>.
- Stripe live integration placeholder is present for production expansion.

13. Security and Data Ownership

- Protected screens redirect unauthenticated users unless guest mode is active.
- API routes call getServerSession and filter records by session.user.id.
- Guest mode is separated from authenticated database users.
- Cart QR payloads are validated with Zod before linking.
- NextAuth secret and database URL are environment variables.

14. Project Structure

- app: pages, layouts, route handlers, and global CSS.

- components: UI, layout, list, cart, and receipt components.
- lib: Prisma client, auth, validation, utilities, hooks, and local product dataset.
- store: Zustand active session store and guest list storage.
- prisma: schema, migrations, and seed data.
- public: images, icons, PWA manifest, and generated service worker files.
- types: shared TypeScript interfaces and NextAuth augmentation.

15. Major Components

- components/carto/QrScanner.tsx
- components/layout/BottomNav.tsx
- components/layout/Header.tsx
- components/layout/Navbar.tsx
- components/layout/PageContainer.tsx
- components/layout/Sidebar.tsx
- components/lists/EditableListTitle.tsx
- components/lists/ListCards.tsx
- components/lists/ListItemsManager.tsx
- components/lists/ProductSearch.tsx
- components/receipt/VirtualReceipt.tsx
- components/ui/Badge.tsx
- components/ui/Button.tsx
- components/ui/Card.tsx
- components/ui/EmptyState.tsx
- components/ui/Input.tsx
- components/ui/LoadingState.tsx
- components/ui/Logo.tsx
- components/ui/MetricCard.tsx
- components/ui/ProgressBar.tsx
- components/ui/ReceiptPanel.tsx

16. NPM Scripts

- dev: next dev
- dev:host: next dev -H 0.0.0.0
- dev:https: next dev --experimental-https -H 0.0.0.0
- tunnel: npx --yes localtunnel --port 3000
- build: next build
- start: next start
- lint: next lint
- db:generate: prisma generate
- db:push: prisma db push
- db:migrate: prisma migrate dev
- db:studio: prisma studio

17. Dependencies

- @prisma/client ^5.19.0
- @stripe/stripe-js ^2.4.0
- @tailwindcss/forms ^0.5.11
- @zxing/browser ^0.1.5
- bcryptjs ^2.4.3
- clsx ^2.1.1
- date-fns ^3.6.0
- dotenv ^17.2.3
- next ^14.2.0
- next-auth ^4.24.7
- next-pwa ^5.6.0
- qrcode ^1.5.3
- qrcode.react ^3.1.0
- react ^18.3.0
- react-dom ^18.3.0
- stripe ^14.21.0
- tailwind-merge ^2.4.0
- zod ^3.23.8
- zustand ^4.5.2

18. Local Setup

- Install dependencies with npm install.
- Configure DATABASE_URL, NEXTAUTH_SECRET, and optionally Stripe keys in .env.
- Generate Prisma client with npm run db:generate.
- Push schema with npm run db:push or run migrations with npm run db:migrate.
- Start development with npm run dev.
- Build production output with npm run build.

19. iPhone Camera Testing

Phone browsers require HTTPS for camera access. A local network HTTP URL can load the app but will block scanning. The project includes HTTPS and tunnel scripts for mobile testing.

- Recommended mobile path: run npm run dev and npm run tunnel, then open the HTTPS tunnel URL on the iPhone.
- Local HTTPS path: run npm run dev:https, open the HTTPS computer IP URL, and trust the local development certificate if iOS asks.
- Desktop localhost camera testing works from http://localhost:3000 on the same computer.

20. Testing and Validation

- Lint command: npm run lint.
- Build command: npm run build.
- Prisma Studio command: npm run db:studio.
- Receipt item scanning can be simulated with POST /api/receipts/:id/items.
- Manual cart linking can be tested with cart ID and pairing code entry.

21. Deployment Notes

- Set production DATABASE_URL, NEXTAUTH_URL, NEXTAUTH_SECRET, and payment keys.
- Run database migrations before serving production traffic.
- Run npm run build and npm start on the hosting platform.
- Use HTTPS in production for auth, PWA behavior, and camera access.

22. Future Enhancements

- WebSocket or server-sent events for true real-time cart updates.
- Production Stripe payment intent integration.
- Store inventory and aisle mapping.
- Advanced analytics and user recommendations.
- Dedicated native mobile app or deeper PWA installation polish.
- Database index review after real usage measurements.